

# CO3 Assessment: Debugging and Optimization – JDBC

## Corrected and Optimized Full Code

```
import java.sql.*;

public class JDBCConnect {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/testdb";
        String user = "root";
        String password = "password";

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            try (Connection con = DriverManager.getConnection(url, user, password)) {
                System.out.println("Connected!");
            }
        } catch (ClassNotFoundException e) {
            System.out.println("MySQL JDBC driver not found.");
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
}
```

## Question 2: SQL Injection Vulnerability

### Corrected and Optimized Full Code

```
import java.sql.*;

public class InsertStudent {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school";
        String sql = "INSERT INTO students(id, name) VALUES (?, ?)";

        try (Connection con = DriverManager.getConnection(url, "root", "pass");
            PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, 1);
            ps.setString(2, "O'Brien");
            int rows = ps.executeUpdate();
        }
    }
}
```

```

        System.out.println("Inserted rows: " + rows);
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}

```

## Question 3: Wrong Parameter Order in PreparedStatement

### Corrected and Optimized Full Code

```

import java.sql.*;

public class UpdateSalary {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/hr";
        String sql = "UPDATE employees SET salary=? WHERE id=?";

        try (Connection con = DriverManager.getConnection(url, "root", "pass");
            PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setDouble(1, 75000.0);
            ps.setInt(2, 101);

            int rows = ps.executeUpdate();
            System.out.println("Updated rows: " + rows);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

## Question 4: Auto-Commit & Missing WHERE Clause

### Corrected and Optimized Full Code

```

import java.sql.*;

public class DeleteRecord {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/inventory";
    }
}

```

```

String sql = "DELETE FROM products WHERE id=?";

try (Connection con = DriverManager.getConnection(url, "root", "pass");
    PreparedStatement ps = con.prepareStatement(sql)) {

    con.setAutoCommit(false);
    ps.setInt(1, 10);

    int rows = ps.executeUpdate();
    con.commit();
    System.out.println("Deleted: " + rows);
} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

## Question 5: ResultSet Column Indexing

### Corrected and Optimized Full Code

```

import java.sql.*;

public class DisplayStudents {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school";

        try (Connection con = DriverManager.getConnection(url, "root", "pass");
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT name, age FROM students")) {

            while (rs.next()) {
                System.out.println(rs.getString(1));
                System.out.println(rs.getInt("age"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

## Question 6: Resource Leak

### Corrected and Optimized Full Code

```
import java.sql.*;

public class FetchEmployees {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/company";

        try (Connection con = DriverManager.getConnection(url, "root", "pass");
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT name FROM employees")) {

            while (rs.next()) {
                System.out.println(rs.getString("name"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

## Question 7: Batch Update Mistake

### Corrected and Optimized Full Code

```
import java.sql.*;

public class BatchInsert {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/store";

        try (Connection con = DriverManager.getConnection(url, "root", "pass");
            Statement st = con.createStatement()) {

            st.addBatch("INSERT INTO items VALUES(1, 'Pen')");
            st.addBatch("INSERT INTO items VALUES(2, 'Pencil')");
            st.addBatch("INSERT INTO items VALUES(3, 'Eraser')");

            int[] result = st.executeBatch();
            System.out.println("Batch executed. Commands: " + result.length);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```
}  
}  
}
```

## Question 8: Transaction Handling Failure

### Corrected and Optimized Full Code

```
import java.sql.*;  
  
public class TransferMoney {  
    public static void main(String[] args) {  
        String url = "jdbc:mysql://localhost:3306/bank";  
  
        try (Connection con = DriverManager.getConnection(url, "root", "pass");  
            PreparedStatement debit = con.prepareStatement(  
                "UPDATE accounts SET balance=balance-? WHERE id=?");  
            PreparedStatement credit = con.prepareStatement(  
                "UPDATE accounts SET balance=balance+? WHERE id=?")) {  
  
            con.setAutoCommit(false);  
  
            debit.setDouble(1, 1000);  
            debit.setInt(2, 1);  
            debit.executeUpdate();  
  
            credit.setDouble(1, 1000);  
            credit.setInt(2, 2);  
            credit.executeUpdate();  
  
            con.commit();  
            System.out.println("Transfer complete!");  
        } catch (SQLException e) {  
            System.out.println("Transfer failed: " + e.getMessage());  
        }  
    }  
}
```

## Question 9: PreparedStatement Reuse Mistake

### Corrected and Optimized Full Code

```
import java.sql.*;

public class InsertMultiple {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/school";
        String sql = "INSERT INTO students(id, name) VALUES(?, ?)";

        try (Connection con = DriverManager.getConnection(url, "root", "pass");
            PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setInt(1, 101);
            ps.setString(2, "Arjun");
            ps.addBatch();

            ps.setInt(1, 102);
            ps.setString(2, "Bhavna");
            ps.addBatch();

            int[] result = ps.executeBatch();
            System.out.println("Inserted rows: " + result.length);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

## Question 10: Wrong Exception Handling & ResultSet Navigation

### Corrected and Optimized Full Code

```
import java.sql.*;

public class SearchProduct {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/shop";
        String sql = "SELECT name FROM products WHERE id=?";

        try (Connection con = DriverManager.getConnection(url, "root", "pass");
```

```

        PreparedStatement ps = con.prepareStatement(sql)) {

    ps.setInt(1, 5);

    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            System.out.println(rs.getString("name"));
        } else {
            System.out.println("Product not found.");
        }
    }
} catch (SQLException e) {
    System.out.println("Database error: " + e.getMessage());
}
}
}

```

## Question 11: Missing Driver & Wrong URL

### Corrected and Optimized Full Code

```

import java.sql.*;

public class ConnectDB {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/test";

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            try (Connection con =
                DriverManager.getConnection(url, "root", "pass")) {
                System.out.println("Connected");
            }
        } catch (ClassNotFoundException | SQLException e) {
            e.printStackTrace();
        }
    }
}

```

## Question 12: ExecuteQuery Misuse

### Corrected and Optimized Full Code

```

Statement st = con.createStatement();
int rows = st.executeUpdate("INSERT INTO emp VALUES(1,'John')");
System.out.println("Inserted rows: " + rows);

```

## Question 13: Hardcoded Credentials

### Corrected and Optimized Full Code

```
String url = System.getenv("DB_URL");
String user = System.getenv("DB_USER");
String password = System.getenv("DB_PASSWORD");

try (Connection con = DriverManager.getConnection(url, user, password)) {
    System.out.println("Connected securely.");
} catch (SQLException e) {
    e.printStackTrace();
}
```

### Optimization / Best Practice

## Question 14: Missing ResultSet.next()

### Corrected and Optimized Full Code

```
ResultSet rs = st.executeQuery("SELECT * FROM emp");

if (rs.next()) {
    System.out.println(rs.getString("name"));
} else {
    System.out.println("No records found.");
}
```

## Question 15: Wrong Column Name

### Corrected and Optimized Full Code

```
ResultSet rs = st.executeQuery(
    "SELECT name AS empname FROM emp");

while (rs.next()) {
    System.out.println(rs.getString("empname"));
}
```



## Question 16: Null Connection Use

### Corrected and Optimized Full Code

```
try (Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/company", "root", "pass");
    Statement st = con.createStatement()) {

    System.out.println("Statement created successfully.");
} catch (SQLException e) {
    e.printStackTrace();
}
```

## Question 17: Duplicate Primary Key Insert

### Corrected and Optimized Full Code

```
String sql = "INSERT INTO emp(id, name) VALUES(?, ?)";

try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setInt(1, 2);
    ps.setString(2, "B");
    ps.executeUpdate();
} catch (SQLException e) {
    System.out.println("Insert failed: duplicate or invalid key.");
}
```

## Question 18: Missing Commit in Transaction

### Corrected and Optimized Full Code

```
con.setAutoCommit(false);

try {
    st.executeUpdate("INSERT INTO emp VALUES(2,'X')");
    con.commit();
    System.out.println("Transaction committed.");
} catch (SQLException e) {
    con.rollback();
    throw e;
}
```

## Question 19: No Rollback on Failure

### Corrected and Optimized Full Code

```
con.setAutoCommit(false);
```

```
try {  
    st.executeUpdate("UPDATE acc SET bal=bal-500 WHERE id=1");  
    st.executeUpdate("UPDATE acc SET bal=bal+500 WHERE id=2");  
    con.commit();  
    System.out.println("Transaction completed.");  
} catch (SQLException e) {  
    con.rollback();  
    System.out.println("Transaction rolled back.");  
    throw e;  
}
```